

PT04 | 02 - Código explicado

Leitura guiada dos trechos reais do App.js

Como ler o App.js

O arquivo é lido de cima para baixo: primeiro entram as ferramentas, depois vem a peça Candidato, em seguida a urna principal e, no final, os estilos.

1. Importações - as ferramentas

```
import React, { useState } from 'react';
import { ScrollView, StyleSheet, Text, TextInput,
  TouchableOpacity, View } from 'react-native';
```

React permite criar componentes. **useState** cria memória que pode mudar. Os itens de react-native são as peças visuais: caixas, textos, campo, botão e rolagem.

2. Componente Candidato - uma peça reutilizável

```
function Candidato({ nome, votos, total, onVotar }) {
  const percentual = total === 0 ? 0 : (votos / total) * 100;
  return (
    <View style={styles.candidatoContainer}>
      <Text style={styles.nomeCandidato}>
        Candidato {nome}: {votos} votos ({percentual.toFixed(1)}%)
      </Text>
      <TouchableOpacity onPress={onVotar}>
        <Text>Votar em {nome}</Text>
      </TouchableOpacity>
    </View>
  );
}
```

A primeira linha recebe as props. A segunda calcula a porcentagem, mas retorna 0 quando ainda não existe voto. O return monta um cartão com texto e botão.

Analogia: é um molde de cartão. O mesmo molde recebe A, B ou C e muda apenas os dados.

3. Estados da urna - cada valor no seu lugar

```
export default function UrnaEletronica() {  
  const [votosA, setVotosA] = useState(0);  
  const [votosB, setVotosB] = useState(0);  
  const [votosC, setVotosC] = useState(0);  
  const [nomeMesario, setNomeMesario] = useState('');
```

export default informa qual componente é a tela principal. As quatro linhas seguintes criam três contadores e o campo do mesário. Cada setter modifica somente o seu valor.

4. Total e reset - regras da urna

```
const totalVotos = votosA + votosB + votosC;  
  
const zerarUrna = () => {  
  setVotosA(0);  
  setVotosB(0);  
  setVotosC(0);  
};
```

O total é uma conta derivada. A função zerarUrna chama os três setters de voto, mas não mexe em nomeMesario; por isso o nome fica preservado.

5. Campo do mesário - valor controlado

```
<TextInput  
  value={nomeMesario}  
  onChangeText={setNomeMesario}  
  placeholder='Digite seu nome'  
/>
```

value mantém o campo sincronizado com o state. onChangeText recebe o texto novo e chama o setter. O operador || mostra Não informado quando o nome está vazio.

6. Botões dos candidatos - callback seguro

```
<Candidato
  nome='A'
  votos={votosA}
  total={totalVotos}
  onVotar={() => setVotosA(prev => prev + 1)}
/>
```

As chaves inserem valores JavaScript no JSX. Para A, o cartão recebe votosA e uma ação que soma um usando prev. B e C repetem a mesma ideia com seus próprios setters.

7. Total e botão de zerar na tela

```
<Text>Total de Votos: {totalVotos}</Text>
<TouchableOpacity onPress={zerarUrna}>
  <Text>Zerar Urna</Text>
</TouchableOpacity>
```

onPress liga o clique à função. Quando ela roda, os setters atualizam o state e o React re-renderiza a tela com os valores novos.

8. Estilos - aparência sem mudar a lógica

```
const styles = StyleSheet.create({
  tela: { flex: 1, backgroundColor: '#F5F5F5' },
  botaoVotar: { backgroundColor: '#0064A0', padding: 14 },
  totalTexto: { fontSize: 20, fontWeight: 'bold' },
});
```

StyleSheet organiza cores, tamanhos, espaços e alinhamento. É a decoração da sala: deixa a urna legível, mas não decide a contagem.

9. Teste de comportamento

```
npm test
npx expo export --platform web
```

O primeiro comando confere a lógica da urna. O segundo verifica se o projeto consegue ser compilado para a web. O teste não substitui abrir a tela no navegador ou no celular.

Como conferir no código

Procure quatro pontos no App.js: **useState** para os valores, **prev => prev + 1** nos votos, **totalVotos** calculado sem state extra e **value/onChangeText** no campo. Esses pontos correspondem diretamente ao que o roteiro pede.